

Hong Hu
hong.hu@berkeley.edu

Executive Summary

This report presents a proof of concept for a Retrieval-Augmented Generation (RAG)-based Q&A system designed to assist with AI project proposal writing, targeting both technical and business (marketing) audiences. The system leverages 78 verified, AI-related questions, employing rigorous prompt engineering and evaluation metrics to generate and assess tailored responses for different user groups.

Introduction

We're tackling the challenge of creating a Proof of Concept for a RAG-based Q&A system to assist with AI proposal writing. The system uses 78 AI-related questions with verified answers, covering topics like neural architectures, training strategies, and fine-tuning techniques. These questions address both technical and business aspects of AI projects, similar to what proposal writers typically encounter. Since proposals require different perspectives for technical and business audiences, we customize answers through prompt engineering, evaluating their accuracy against ground truth for both engineers and marketing teams.

Key Findings

Here's a summary of the main takeaways from building our RAG-based Q&A system:

- Fine-tuning hyperparameters is key for getting the best results from the system. I have listed all available options for each hyperparameter in Appendix A. The most important variables are loader, embedding model, reranker choice, and GenAI/LLM models.
- While standard cosine similarity is a useful baseline, it doesn't capture subtle differences between responses. For a more accurate assessment, we need to use LLM scoring and check Context Relevance.
- Mistral is a free LLM, but its performance is lacking, and it often fails to return valid JSON, which is critical for the reranker. Cohere is a much stronger LLM, though it isn't free, making it a good option for formal validation. Mistral, while slow, is still suitable for experimentation and parameter tuning due to its open availability.
- Some rerankers based on the traditional TF-IDF algorithm^[4] are much faster than LLM-driven rerankers, but the accuracy isn't as high. We'll have to strike a balance between speed and accuracy for real-world deployment.
- Simple toy examples aren't enough for choosing the right embedding model—you really need to test them on actual data to judge their effectiveness.

Methodology

Since there are 13 settings ("knobs") to adjust, there are a total of 960 possible options—making a full grid search too costly. To make hyperparameter optimization for our RAG-based Q&A system more cost-effective, I chose open-source Mistral as the primary LLM for both reranking and generating answers. I began experiments with a small validation dataset. After finding a close-to-optimal set of hyperparameters, I moved on to use the full sample set. The data is loaded, chunked, and stored in Qdrant. Then, we use RAG for retrieval, rerank the results, and generate answers using the LLM. We evaluate performance using RAGAS and OpenAI. Low-scoring samples are further analyzed to refine the hyperparameters. In the final setup, I upgrade from Mistral to the more advanced commercial Cohere model.

Technical Approach

Based on the methodology, here are the technical choices I made:

- **Loader:** I implemented a smart loader that uses regular expressions to remove all content after the "References" section. This keeps the data clean and relevant for the Q&A system, and helps us avoid the need to tune prompts specifically to filter out reference material during the reranking stage.

- **Chunker:** I also used regular expressions to split the documents by paragraphs instead of by characters, resulting in more meaningful chunks. For chunk size, I set a maximum of 128 characters. Using smaller chunks can reduce the context available to the LLM, leading to less coherent answers, while larger chunks could introduce unnecessary noise. I chose zero overlap between chunks to prevent redundancy and confusion during retrieval. Additionally, I filter out any chunks containing fewer than 10 words to ensure that each chunk has sufficient information for the LLM to generate a meaningful response.
- **Embedding Model:** I selected the "all-distilroberta-v1" model for embeddings, since it delivered the highest RAGAS evaluation scores on our 78 validation samples.
- **Retrieval:** I chose to retrieve the top 10 candidates from the Qdrant vector database. This number balances the need to have a diverse set of candidate answers while keeping the computational cost of reranking manageable.
- **Reranker:** I switched to the Cohere model for reranking candidates since it's more advanced than Mistral and performed better in our RAGAS evaluations. Mistral often gave formatting issues, like missing keys in the JSON output, which required a lot of manual correction. I then selected the top 3 reranked candidates for answer generation, which reduces token usage and gives the answer generation model enough context to produce strong answers.
- **LLM:** I also used Cohere for answer generation, as it reliably follows instructions tailored for both engineering and marketing teams. Using one LLM for both reranking and generation also simplifies the system architecture.
- **Prompt Engineering:** I designed a prompt template that includes the question, the top 3 reranked chunks, and specific instructions to generate answers suitable for both technical and business audiences.
- **Addressing Two Audiences:** Since we're often missing context, we can't just rely on Wikipedia for marketing or limit research responses to papers and web sources. Instead, I decided to use the LLM's capabilities to tailor answers for both groups, making sure all available context is leveraged.

Testing and evaluation

I use composite and hierarchical evaluation metrics to evaluate the RAG model. The final score are composed of two major parts: Similarity and RAG Context Relevance, each occupying 50% of the total score. The Similarity part is further divided into two sub-metrics: Cosine Similarity and LLM Judge, each occupying 25% of the total score as demonstrated in Fig 1. Cosine Similarity, LLM Judge and RAG Context Relevance are equivalent to $SemanticSimilarity^{[1]}$, $AnswerAccuracy^{[2]}$ and $LLMContextPrecisionWithoutReference^{[3]}$ in RAGAS with OpenAI GPT-4o-2024-08-06 model. I also provide two special similarity score to measure the performance of Engineering and Marketing answers separately.

$$\alpha = \text{Cosine Similarity} = \frac{\sum_{i=0}^N \frac{SemanticSimilarity_{research} + SemanticSimilarity_{marketing}}{2}}{N}$$

$$\beta = \text{LLM Judge} = \frac{\sum_{i=0}^N \frac{AnswerAccuracy_{research} + AnswerAccuracy_{marketing}}{2}}{N}$$

$$\gamma = \text{Context Relevance} = \frac{\sum_{i=0}^N LLMContextPrecisionWithoutReference}{N}$$

$$Score = \frac{1}{4}\alpha + \frac{1}{4}\beta + \frac{1}{2}\gamma$$

Fig.1: Formula of the Evaluation Metrics

The Similarity introduced both classical cosine similarity as well as LLM judge to comprehensively evaluate the closeness between ground truth and generated answer. The RAG Context Relevance is in 50% of the total score because a lot of questions cannot be answered by the context. For example, questions 24-30 are all related to

Chinchilla, which is a topic that is not covered in the provided documents. The answers to these questions are purely based on the LLM's pretraining or supervised fine-tuning (SFT). Therefore, evaluating the context relevance is more important than just the accuracy of the answer.

My test runs were mainly conducted to see how adjusting different settings would affect performance, so I could identify the best combination of hyperparameters for the highest overall score.

Results and Findings

Proof of Concept Functionality

The proof of concept clearly shows that our RAG-based Q&A system works as intended for AI proposal writing. The pipeline does a solid job of retrieving relevant information, reranking the candidates, and generating customized answers for both technical and business users. We thoroughly tested the system to validate its performance. Appendix B provides a snapshot of the results, evaluated using LangSmith and compared against other models with small adjustments—demonstrating that our pipeline delivers better outcomes following several rounds of improvements.

Lessons Learned

I invested too much effort in trying to refine the reranker prompt to filter out reference sections, but it was not very effective. In retrospect, applying regular expressions to remove references from the dataset before the reranking process would have made things much simpler and boosted performance. I also should have reviewed the validation set in advance to spot questions, like those about Chinchilla, that aren't covered in our documents. If I had done that earlier, I could have designed the composite evaluation metrics sooner.

Challenges and Limitations

The system's capabilities are tied to how good and relevant the provided literature is. Also, the RAG model relies on both the retriever and reranker, which don't always guarantee the best results. Since there are only 78 validation samples, it's tough to fully judge how well the model works on a wide range of questions. Because we can't run the reranker, LLM, and evaluation in parallel in Jupyter, things might run slower and the overall workflow could feel less efficient, which may impact user satisfaction.

Next Steps

If I had more time, I would focus on fine-tuning the LLM to improve the model's performance. I would want to use a dataset that includes a wide range of AI-related topics, including both technical and non-technical aspects. The training approach would involve supervised fine-tuning with a focus on question-answer pairs, ensuring that the model learns to generate accurate and contextually relevant responses. Reranking model can also be trained to enhance the model's ability to select the most relevant answers from a set of candidates. This would involve using a dataset with multiple candidate contexts for each question, allowing the model to learn to rank them based on relevance and accuracy. Additionally, incorporating user feedback into the training process could help the model adapt to different user personalities and preferences, further improving its performance in real-world applications. The current pipeline is also running in serial mode, which is not optimal for mixture of experts (MoE) models. I would like to explore how to run the pipeline in parallel mode to improve the efficiency of the system.

Summary & Recommendations

Using LangSmith for visualization really saved me a lot of time, allowing me to focus more on running experiments. TF-IDF performed quite well: even though it doesn't outperform the Cohere LLM reranker, its results are comparable to those of the current Mistral LLM ranker. TF-IDF is also much faster and helps reduce LLM usage costs. We could offer a "fast mode" option powered by TF-IDF for users who prefer quicker, though slightly less accurate, responses. It might also be worthwhile to test other similar reranking algorithms, like md25. Additionally, saving temporary results to Qdrant instead of keeping them in memory would make debugging easier, since you wouldn't have to restart everything from scratch.

References

[1] Semantic similarity: https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/semantic_similarity/

[2] Answer Accuracy: https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/nvidia_metrics/

[3] Context Precision: https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/

[4] TfidfVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

Appendix A: All Available Options for Each Hyperparameter

```
{
  "load_strategy": ["raw", "smart"],
  "regex_param": ["paragraph"],
  "chunk_size": [50, 128, 500, 5000],
  "chunk_overlap_pct": [0, 0.1, 0.2],
  "chunk_strategy": ["only", "filter"],
  "embed_model": ["multi-qa-mpnet-base-dot-v1", "all-mpnet-base-v2", "all-MiniLM-L12-v2", "all-distilroberta-v1", "multi-qa-distilbert-cos-v1"],
  "rag_top_n": [10],
  "test_data": ["experiment", "validation"],
  "rerank_strategy": ["pass", "tfidf", "llm-reranker"],
  "rerank_top_n": [3],
  "llm": ["mistral"],
  "prompt": ["generic", "custom"],
  "is_reset": [false]
}
```

Appendix B: Evaluation with LangSmith

I evaluated the optimal pipeline, identified as (*) Cohere Roberta, against other models, making minor adjustments to the hyperparameters. The model in the Jupyter notebook outperformed the alternatives, achieving a higher score. Please see the snapshot below for reference.

- (*) Cohere Roberta: Current Pipeline
- Mistral Generic Roberta: Mistral using only one prompt template
- Mistral Mpnet: Mistral utilizing multi-qa-mpnet-base-dot-v1
- Mistral No-rerank Roberta: Mistral without a reranker
- Mistral TFIDF Roberta: Mistral with a TFIDF reranker

